

Contents

1	Methods and Experimental Setup	3
1.1	Research Design and Approach	3
1.2	Anomaly Detection Algorithms	4
1.2.1	One-Class SVM	4
1.2.2	Isolation Forest	5
1.2.3	Feedforward (MLP) Autoencoder	5
1.2.4	LSTM Autoencoder	5
1.2.5	Predictive LSTM	6
1.2.6	Transformer-Based Autoencoders	6
1.3	Datasets	7
1.3.1	Credit Card Fraud Detection (Dataset A)	7
1.3.2	Yahoo Webscope S5 (Dataset B)	8
1.3.3	CIC-IDS2017 (Dataset C)	8
1.4	Preprocessing	9
1.4.1	Tabular Preprocessing (Credit Card)	9
1.4.2	Time-Series Windowing (Yahoo S5)	10
1.4.3	Network Flow Preprocessing (CIC-IDS2017)	11
1.5	Hyperparameter and Training-Related Choices	12
1.5.1	Isolation Forest	12
1.5.2	One-Class SVM	13
1.5.3	Feedforward (MLP) Autoencoder	13
1.5.4	Predictive LSTM	13
1.5.5	LSTM Autoencoder	14
1.5.6	Bottleneck Transformer Autoencoder	14
1.5.7	Causal Transformer	14
1.6	Evaluation Protocol	15
1.6.1	Metrics Reported	15
1.6.2	Threshold Selection	15
1.6.3	No Point Adjustment	16

2	Experimental Results	17
2.1	Credit Card Fraud Detection (Dataset A)	17
2.1.1	Overall Comparison	17
2.1.2	Discussion	17
2.2	Yahoo Webscope S5 (Dataset B)	19
2.2.1	A1 – Point Anomalies (Real Server Metrics)	19
2.2.2	A3 – Contextual/Seasonal Anomalies	19
2.2.3	A4 – Level-Shift Anomalies	20
2.2.4	Trivial Baseline Sanity Check (Yahoo Only)	21
2.2.5	Discussion	22
2.3	CIC-IDS2017 (Dataset C)	23
2.3.1	Overall Comparison	23
2.3.2	Per-Attack-Type Breakdown	26
2.3.3	Discussion	28
2.4	Exploratory Data Analysis	28
2.4.1	Credit Card Fraud Detection	29
2.4.2	Yahoo Webscope S5	30
2.4.3	CIC-IDS2017	31
2.5	Explainability	32
3	Explainability Case Studies	33
3.1	Reconstruction-Error Localization on Yahoo A3	33
3.1.1	Initial Assumption	33
3.1.2	Qualitative Check: Per-Timestep Error Heatmaps	34
3.1.3	Quantitative Verification: Gini Coefficient of Error Concentration	35
3.2	Pseudo-Sequence Reconstruction Error on Credit Card Fraud	36
3.2.1	Initial Assumption	37
3.2.2	Qualitative Check: Per-Feature-Position Error Heatmaps	37
3.2.3	Quantitative Verification: Column-Shuffle Feature-Identity Check	39
3.3	Disjoint Feature Regimes: AE+Context vs. Temporal IForest	40
3.3.1	Temporal IForest: Initial Assumption	40
3.3.2	Temporal IForest: Score Validity Under Last-Flow Labeling	40
3.3.3	Temporal IForest: SHAP Attribution Across the Window	41
3.3.4	AE+Context: SHAP Dominated by Host-Context Features	42
3.3.5	Building the Combined Score	44
3.3.6	Two Fusion Rules, Two Normalizations	44
3.3.7	Per-Class Coverage vs. Aggregate AP	45

Chapter 1

Methods and Experimental Setup

1.1 Research Design and Approach

This project is organized around a single question: how does the way a model represents the *latent structure* of "normal" data determine what kinds of anomalies it can and cannot detect? Rather than optimizing one architecture for one dataset, the approach taken here is deliberately comparative along two axes at once.

The first axis is **paradigm**: the seven models evaluated (§1.2) were chosen specifically to cover four structurally distinct ways of representing what "normal" looks like – a **boundary** carved out in kernel space (One-Class SVM), a **partitioning** of the feature space by random splits (Isolation Forest), a learned **compression bottleneck** that must discard everything but the dominant structure of normal data (the three reconstruction-based autoencoders), and a **predictive** model of how normal data evolves one step at a time (Predictive LSTM, Causal Transformer). These are not arbitrary choices or a convenience sample of popular architectures: each paradigm encodes a different implicit assumption about what makes a point anomalous, and the central empirical question throughout this thesis is which assumption matches which kind of anomaly.

The second axis is **dataset**: Credit Card Fraud, Yahoo Webscope S5, and CIC-IDS2017 (§1.3) were selected to span a deliberate spectrum of temporal structure – from a purely tabular dataset with no meaningful ordering between rows, through genuinely sequential time series, to network flow data that admits flat, temporal, and host-context views of the same underlying events. Running every model paradigm against every point on this spectrum is what makes it possible to attribute a given result to the *paradigm* (does this kind of anomaly need a predictive objective, or a bottleneck, or neither?) rather than to some idiosyncrasy of a single dataset.

The resulting experimental method follows a fixed three-stage cycle, repeated per dataset in Chapter 2:

1. **Measure.** Train every applicable model exclusively on benign data and evaluate

all of them under the same protocol (§1.6), producing a ranked comparison table and, where the data supports it, a breakdown by anomaly type or attack class.

2. **Explain.** Read the resulting performance gaps – not just which model wins, but which paradigm wins where – and propose a concrete mechanistic explanation grounded in each model’s architecture (e.g. “the LSTM Autoencoder’s pooled hidden state destroys the phase information a seasonal anomaly needs”).
3. **Verify.** Treat that explanation as a falsifiable claim and test it directly with a targeted diagnostic – reconstruction-error localization, SHAP attribution, or a controlled perturbation of the input – rather than letting the aggregate metrics alone stand as evidence for the mechanism. Chapter 3 is dedicated entirely to this stage, and its findings are reported honestly as confirming, complicating, or refining the claims made in Chapter 2, not merely as illustrations of them.

This cycle is what separates the thesis’s contribution from a plain benchmark comparison: the goal is not only to report which of seven models scores highest on which of three datasets, but to build and test an explanation of *why*, at the level of paradigm rather than individual model.

1.2 Anomaly Detection Algorithms

All models below are used for *novelty detection*, not outlier detection: each is fit exclusively on benign/normal data and never sees an anomaly during training, so “normal” is whatever the training distribution looked like. At test time each model produces a scalar anomaly score per sample (never a hard label directly), and the label is obtained by thresholding that score with the procedure in §1.6. The seven models fall into four paradigms: **boundary-based** (One-Class SVM), **partition-based** (Isolation Forest), **reconstruction-based** (Feedforward Autoencoder, LSTM Autoencoder, Bottleneck Transformer Autoencoder), and **temporal-predictive** (Predictive LSTM, Causal Transformer).

1.2.1 One-Class SVM

One-Class SVM (`sklearn.svm.OneClassSVM`, ν -SVM formulation) learns a boundary around the training data in a (typically infinite-dimensional) feature space induced by a kernel, rather than reconstructing or partitioning the input directly. With the RBF kernel used throughout this project, the model implicitly fits a smooth, closed decision region around the bulk of the benign training points. Fixed across all three datasets: `kernel='rbf'`, `nu=0.01`, `gamma='scale'` (see §1.5). At test time, `decision_function` returns the signed distance to the learned boundary (positive inside, negative outside);

this is negated so that, consistently with every other model in this project, *higher score = more anomalous*.

1.2.2 Isolation Forest

Isolation Forest (`sklearn.ensemble.IsolationForest`) is partition-based: it builds an ensemble of random trees that recursively split the feature space on randomly chosen features and split points, and uses the average path length required to isolate a point (into its own leaf) across all trees as the anomaly signal. Because splits are random, points that are easy to separate from the rest of the data – i.e. anomalies, which tend to sit in sparse, low-density regions – are isolated in fewer splits, giving them a shorter average path length than typical benign points. `score_samples` returns the negative mean path length, which is negated once more here so that higher score means shorter path (i.e. more anomalous), keeping the convention consistent with every other model. `contamination` is left at its default, 'auto': this parameter would normally set the model's own internal decision threshold based on an assumed anomaly fraction in the training data, but since training data here is exclusively benign (no contamination to speak of) and the operating threshold is instead selected externally via the PR-curve sweep in §1.6, `contamination` has no effect on the reported results – only the continuous anomaly score matters.

1.2.3 Feedforward (MLP) Autoencoder

The Feedforward Autoencoder is reconstruction-based: a symmetric encoder/decoder pair of fully-connected `Linear`+`ReLU` layers (with dropout between hidden layers) compresses the input feature vector down through a bottleneck of strictly reduced dimensionality and then expands it back to the original input size, trained end-to-end to minimize mean squared reconstruction error against its own input. Because the bottleneck is narrower than the input, the network cannot simply learn the identity function; it is forced to learn a compressed representation that captures the dominant structure of *benign* data specifically. At test time, the anomaly score is the per-sample mean squared error between the input and its reconstruction: benign samples, which resemble the training distribution the bottleneck was shaped around, reconstruct with low error, while anomalous samples – which the bottleneck was never trained to represent – tend to reconstruct poorly. Layer widths and the resulting bottleneck ratio vary by dataset (§1.5) but the encoder/decoder-mirror-of-encoder architecture is identical everywhere.

1.2.4 LSTM Autoencoder

The LSTM Autoencoder applies the same reconstruction principle as the Feedforward Autoencoder, but to sequence-shaped input: one LSTM layer serves as the encoder, con-

suming the full input sequence and retaining only its final hidden (and cell) state as a single fixed-size vector summary of the entire sequence. That final hidden state is then repeated identically at every timestep as the input to a second LSTM layer acting as the decoder, whose per-timestep outputs are projected back to the original feature dimensionality by a shared linear layer. The anomaly score is again per-sample mean squared error, computed over every timestep and feature jointly between the input sequence and its reconstruction.

1.2.5 Predictive LSTM

The Predictive LSTM uses the same recurrent building block (a single LSTM layer) as the LSTM Autoencoder above, but with a categorically different training objective: rather than reconstructing its own input, it is trained to predict step $t + 1$ from steps $0..t$ at every position in the sequence simultaneously – the model never sees the value it is being asked to predict. Architecturally, this means the LSTM’s hidden state at each timestep is projected directly to a next-step prediction via a linear output layer, with no encoder/decoder split and no bottleneck vector: the model consumes the sequence once, and every hidden state along the way is used, rather than only the final one. The anomaly score is the mean squared error between predicted and true next-step values across the sequence. This distinction in *objective* (predict the future vs. reconstruct the present), rather than any difference in raw architectural capacity, is what separates this model from the LSTM Autoencoder, and turns out to matter substantially more than the choice of recurrent vs. attention-based backbone for how each model responds to different anomaly types (§2).

1.2.6 Transformer-Based Autoencoders

Two transformer variants are used, both replacing the LSTM Autoencoder’s recurrent backbone with a multi-layer `TransformerEncoder` (self-attention + feedforward blocks, sinusoidal positional encoding, identical `D_MODEL/NHEAD/NUM_LAYERS/DIM_FF` across datasets – §1.5), but differing in how the bottleneck (reconstruction) or causality (prediction) constraint is imposed:

- **Bottleneck Transformer Autoencoder** – reconstruction-based. Each input feature/timestep is projected to a `D_MODEL`-dimensional token and passed through the transformer encoder with full (non-causal) self-attention, so every position can attend to every other position freely. The resulting per-position representations are then *global-average-pooled* across the sequence into a single `D_MODEL`-dimensional vector, squeezed through a linear layer to a strictly smaller `LATENT_DIM`, and expanded back out by a second linear layer to the full flattened sequence shape. This

pool-squeeze-expand step is the explicit information bottleneck (analogous to the LSTM Autoencoder’s final hidden state) that prevents the attention mechanism from simply copying its input through to the output. Anomaly score: per-sample MSE between input and reconstruction across all positions, computed in a single forward pass.

- **Causal Transformer** – temporal-predictive, the attention-based counterpart to the Predictive LSTM. The same encoder stack is used, but with a causal (upper-triangular) attention mask applied, so position t can only attend to positions $0..t$; the model is trained on the same next-step objective as the Predictive LSTM, with no pooling bottleneck. Anomaly score: per-sample MSE between predicted and true next-step values.

Both variants deliberately use a single `TransformerEncoder` stack rather than the full encoder-decoder structure common in the time-series transformer literature (e.g. Informer’s ProbSparse-attention encoder paired with a generative decoder, or reconstruction-based designs that pair a Transformer encoder with a separate convolutional decoder). Training and inference cost scale with the number of attention layers on both datasets and models, and an encoder-only design keeps that cost, along with the number of trainable parameters, to roughly half of what an equivalent encoder-decoder model would require, without sacrificing the ability to either pool-and-reconstruct or causally predict.

No Causal Transformer implementation exists for the credit card dataset (§2).

1.3 Datasets

Three datasets were selected to span a deliberate spectrum of temporal structure: Dataset A is purely tabular, with no meaningful temporal ordering between rows; Datasets B and C are genuinely temporal, with Dataset C additionally admitting both a flat/tabular and a temporal view of the same underlying data. This lets the same four modeling paradigms be evaluated under progressively harder conditions for the “does temporal order matter” question.

1.3.1 Credit Card Fraud Detection (Dataset A)

The Kaggle Credit Card Fraud Detection dataset contains 284,807 transactions, of which 492 (0.172%) are fraudulent – an extreme class imbalance typical of real-world fraud detection. Each row has 30 features: **Time** (seconds elapsed since the first transaction in the dataset), 28 PCA-transformed features **V1–V28** (anonymized for confidentiality, already decorrelated and roughly zero-mean by construction), and **Amount** (raw transaction value, heavily right-skewed).

The dataset, along with community exploratory analysis, is available on Kaggle.¹ A further exploratory notebook specifically covering imbalanced-data handling and anomaly detection on this dataset is also available.²

Time is elapsed seconds, not a repeating or structured temporal axis – there is no periodicity, trend, or session concept to exploit. This makes Dataset A the deliberate “no genuine temporal order” control case.

1.3.2 Yahoo Webscope S5 (Dataset B)

Yahoo Webscope S5 consists of four benchmark families (A1–A4), each a collection of independent univariate time series with labeled anomalies, but each constructed to isolate a different anomaly type:

Benchmark	Series	Origin	Anomaly type	Anomaly rate
A1	67	Real server metrics	Point outliers	1.76%
A2	100	Synthetic, single periodic component + trend	Point outliers	0.33%
A3	100	Synthetic, 3 interacting frequency components	Contextual (phase deviation)	0.56%
A4	100	Synthetic, same structure, no seasonality	Level shifts	0.50%

A1 is real, organic, and noisy (messy daily/weekly cycles); A2–A4 are synthetic and purpose-built so that the anomaly type is controlled independently of the underlying series shape. This is what allows the Results chapter to attribute performance differences to *anomaly type* (point vs. contextual vs. level-shift) rather than to confounding differences between datasets. A2 is included in the codebase but has only a partial run so far (see §2).

1.3.3 CIC-IDS2017 (Dataset C)

CIC-IDS2017 (Canadian Institute for Cybersecurity) is a network intrusion detection benchmark captured over five working days: ~2.8 million bidirectional network flows, each described by ~80 statistical features extracted by CICFlowMeter (duration, packet/byte counts and rates, inter-arrival-time statistics, TCP flag counts, window sizes, etc.). Monday is 100% benign traffic; 14 distinct attack types (DoS/DDoS variants, PortScan, brute-force, web attacks, Bot, Infiltration, Heartbleed) are injected Tuesday through Friday. The Monday-benign / rest-mixed split is therefore a natural, dataset-native novelty detection split.

The raw/Kaggle distribution of this dataset is known to be substantially corrupted (Engelen et al., WTMC 2021): roughly 26% of flows are TCP “appendix” artifacts inheriting their parent flow’s attack label without containing any attack traffic, a large fraction

¹<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

²<https://www.kaggle.com/code/janiobachmann/credit-fraud-dealing-with-imbalanced-datasets>

of nominally-labeled attack flows carry no actual payload (label-only, not behavior-based), and the entire DoS Hulk class corresponds to a broken attack simulation that never executes against the target server. Identifier columns (**Flow ID**, source/destination IP, **Timestamp**) are also present and, if kept, let a model shortcut-learn by IP address or capture time window instead of by traffic behavior. This project uses the cleaned, re-extracted version of the dataset from Engelen et al. rather than the original Kaggle/UNB CSVs, and drops the identifier columns explicitly (see §1.4).

CIC-IDS2017 flows are session-level statistical aggregates. This makes the dataset a genuine bridge case: it supports a valid flat/tabular view (one flow = one sample, analogous to Dataset A), a valid temporal view (flows grouped and ordered per host, analogous to Dataset B), and, as introduced in this project, a host-context view that augments each flow with rolling per-host activity statistics, adapted from the unsupervised method of Raskovalov et al. (2024) and described in full in §1.4. Comparing all three feature-engineering strategies against the same underlying data and the same model paradigms is the core experimental design for Dataset C.

1.4 Preprocessing

Two principles are shared across all three pipelines. First, every model is trained exclusively on benign/normal samples (novelty detection): the training split never contains an anomaly, and all scalers/normalizers are *fit* on the training split only, then applied unchanged to the test split – this prevents any statistic of the anomalous test data from leaking into preprocessing. Second, the test split always contains both classes, evaluated with the protocol described in §1.6.

1.4.1 Tabular Preprocessing (Credit Card)

1. **Time** is dropped (not a meaningful axis, see §1).
2. Rows are split by class: all benign (**Class**=0) rows are pooled, then split 80/20 (**TEST_SIZE**=0.2, **RANDOM_SEED**=123) into train and test-benign. The training set is the 80% split, containing only benign rows (~227,451 rows).
3. The test set is the remaining 20% benign rows plus *all* 492 fraud rows.
4. Scaling is asymmetric by feature group: **V1--V28** pass through **unscaled** by default (**V_SCALING** = 'none' – reasonable since these are already PCA-whitened features), with 'standard' also attempted on some runs but no real difference across models. **Amount** is scaled independently with a **RobustScaler** (median/IQR-based), chosen

for robustness to its heavy right tail, following the same reasoning as the exploratory notebook cited in §1.³

Output: `X_train_normal.npy` ($\sim 227k \times 29$), `X_test.npy` / `y_test.npy` (mixed class), under `data/processed_data/processed_credit_card/`.

1.4.2 Time-Series Windowing (Yahoo S5)

Implemented in `yahoo/preprocess_timeseries.py`, run once per benchmark (the `BENCHMARK` constant is edited and the script re-run for each of A1/A2/A3/A4, producing one `processed_yahoo/{Ben` directory per run).

1. Each series is split 70/30 along the time axis (`TRAIN_FRAC=0.70`) – a temporal split, not a random shuffle, so no future information leaks into training.
2. A `MinMaxScaler` is fit *per series*, using only the non-anomalous points of that series’ training portion, then applied to both the train and test portions of the same series.
3. A sliding window of length `WINDOW=64` is applied: stride 4 for training windows (`STRIDE_TRAIN`, subsampled, and only fully-normal windows are kept for training) and stride 1 for test windows (`STRIDE_TEST`, dense – every possible window position is evaluated).
4. A window’s label follows the “any” rule: positive if at least one of its 64 timesteps is anomalous. This is a documented source of label blur – e.g. on A1, a 1.76% point-level anomaly rate becomes a $\sim 17\text{--}18\%$ window-level rate, since a single anomalous point contaminates up to 64 overlapping windows. A smaller window would reduce this blur; re-running with $w = 35$ (following the DeepAnt window-size ablation discussed below) is worth trying, though this has not yet been done.
5. Windows from all series in a benchmark are pooled into single `X_train_normal.npy` / `X_test.npy` / `y_test.npy` arrays.

Comparison with DeepAnt. DeepAnt (Munir et al., 2019) was used as a reference for evaluating a deep next-step predictor on Yahoo S5, though several methodological choices differ from this project’s:

- **Many-to-one prediction, not many-to-many.** DeepAnt’s proposed predictor is a convolutional network; the LSTM variant they report as a comparison baseline (two stacked layers of 32 units each) is narrower and shallower than the single 64-unit LSTM used for Predictive LSTM here. More importantly, both predict only

³<https://www.kaggle.com/code/janiobachmann/credit-fraud-dealing-with-imbalanced-datasets>

a single next timestep from a window (many-to-one), rather than every step $t+1$ from $0..t$ simultaneously, as this project’s predictive models do (§1).

- **Per-series vs. pooled training.** DeepAnt fits and evaluates a separate model on each individual time series (a 40/60 train/test split per series, F-score computed per series and then averaged per sub-benchmark), which lets each model specialise to a single frequency. This project instead pools windows from all ~ 100 series in a benchmark into one shared model trained on a 70/30 split – mostly for ease of implementation.

1.4.3 Network Flow Preprocessing (CIC-IDS2017)

Three preprocessing variants exist for this dataset, all sharing the same base cleaning steps (drop `Flow ID`/`Src IP`/`Dst IP`/`Timestamp`/`Src Port`; one-hot encode `Protocol`; replace `Inf` with `NaN`; median-impute using training-set medians; drop zero-variance columns measured on the training set; `MinMaxScaler` fit on the training set only) but differing in unit of observation:

Flat (`preprocess.py`). One row = one flow. Training set is all Monday benign flows (371,749 rows); test set is all Tuesday–Friday benign flows plus all real attack flows (-- `Attempted` flows, i.e. attacks with no observed payload, are excluded entirely). Zero-variance filtering on Monday drops the three URG-flag columns, leaving 77 of the original 80 features.

Temporal (`preprocess_temporal.py`). One row = a window of `WINDOW=16` consecutive flows from the same source IP, sorted by timestamp. This grouping choice – by source IP rather than by individual 5-tuple flow session – is deliberate and matches the grouping key already used to compute the host-context features above (`preprocess_context.py`): both pipelines treat "one entity" as one source host’s ordered activity stream, not one 5-tuple connection, since it is host-level behaviour (many short connections in a burst) rather than any single connection’s internal structure that characterises attacks such as PortScan or DoS. Grouping by source IP works because the ~ 48 testbed hosts each generate thousands of ordered flows; at `WINDOW=16`, source-IP streams cover 100% of flows for 12 of 14 attack classes. The split is day-based (Monday train, Tuesday–Friday test) for the same reason as the flat pipeline; an earlier position-based (80/20 row-index) split was tried and abandoned after it was found to silently discard entire attack classes whose flows were concentrated late in the row order (documented as a fixed bug in `mds/cicids2017.md`). Output shape: (N, 16, 77).

Host-context (`preprocess_context.py`). Same flat, per-flow cleaning as above, but each flow is additionally augmented with 9 engineered features computed from a 60-second sliding time window per host IP: flow count, unique port count, port-diversity rate, a new-port indicator, and the source/destination asymmetry of each. These are computed causally (only from activity *prior* to the current flow) and require no attack labels, following the unsupervised adaptation of Raskovalov et al. (2024). Final feature count: $77 + 9 = 86$. This variant was introduced specifically to address a blind spot shared by both the flat and temporal pipelines on the PortScan attack class (see §2).

1.5 Hyperparameter and Training-Related Choices

Two policy choices apply uniformly across all models and datasets:

- **Optimizer.** All neural networks are trained with Adam, at a fixed learning rate of 10^{-3} , uniformly across every model and dataset.
- **No hyperparameter sweeping.** If a model performed unexpectedly poorly, its learning rate was sanity-checked against $\{10^{-4}, 10^{-3}, 3 \times 10^{-3}\}$ before drawing any conclusion from that result.
- **Validation split for monitoring only.** Every neural network holds out a random 10% of its training data purely to plot a training/validation loss curve per run, as a sanity check that training is proceeding correctly. This split is never used for early stopping or model selection: training always runs for the fixed epoch count below, and the reported test-set results use the model’s final-epoch weights.

1.5.1 Isolation Forest

	CIC flat	CIC context	Yahoo	Credit card
N_ESTIMATORS	150	150	150	150
MAX_SAMPLES	300,000	300,000	$\min(n, 50,000)$	200,000
Training set size	~371k	~371k	few thousand	~227k
% of train used	~81%	~81%	~100%	~88%
RANDOM_SEED	123	123	123	123

MAX_SAMPLES is the only meaningful difference and is set as high as practically feasible per dataset (accuracy improves with more samples per tree, with diminishing returns; swamping the trees is not a concern here since training data is always benign-only). Yahoo’s 50,000 cap is never actually reached and is kept only as a safety ceiling.

1.5.2 One-Class SVM

	CIC flat	CIC context	Yahoo	Credit card
NU	0.01	0.01	0.01	0.01
KERNEL	rbf	rbf	rbf	rbf
GAMMA	scale	scale	scale	scale
SUBSAMPLE	50,000	50,000	none (full)	none (full)
% of train used	~13.5%	~13.5%	~100%	~100%

All kernel/nu parameters are uniform across datasets. Subsampling is applied only where the training set is large enough to make full-data OC-SVM impractically slow; credit card (~227k rows) currently trains on the full set without subsampling.

1.5.3 Feedforward (MLP) Autoencoder

	CIC flat	CIC context	Yahoo	Credit card
Input dim	77	86	64 (window flattened)	~30 (PCA)
Hidden dims	[64, 32, 16]	[64, 32, 16]	[32, 16]	[16, 8]
Bottleneck ratio	0.21	0.19	0.25	~0.27
Final activation	Sigmoid	Sigmoid	Linear	Linear

Layer widths scale with input size to hold the bottleneck compression ratio roughly constant (~0.20–0.27) across datasets. The final activation differs because CIC uses `MinMaxScaler` (bounded to $[0, 1]$, matching a sigmoid output), while Yahoo and credit card use unbounded scalers (linear output).

1.5.4 Predictive LSTM

	CIC	Yahoo	Credit card
INPUT_SIZE	77	1	1
HIDDEN_DIM	128	64	64
NUM_LAYERS	1	1	1
EPOCHS	20	20	50
BATCH_SIZE	256	256	512

HIDDEN_DIM scales with input size for CIC (a $\sim 1.66\times$ expansion over 77 input features); both univariate datasets use 64, sufficient capacity for a scalar signal. Credit card uses more epochs (50) because its loss curve keeps declining past epoch 30 – a direct consequence of treating 29 non-temporal PCA features as a pseudo-sequence, which makes optimization noisier than on genuinely temporal data. CIC and Yahoo converge by epoch ~ 5 , so 20 epochs is already conservative for them.

1.5.5 LSTM Autoencoder

	CIC	Yahoo	Credit card
INPUT_SIZE	77	1	1
HIDDEN_DIM	128	64	64
NUM_LAYERS	1	1	1
EPOCHS	50	30	50
BATCH_SIZE	256	256	512

CIC and credit card both required 50 epochs to plateau; Yahoo converges by epoch ~5, so 30 epochs is already generous there.

1.5.6 Bottleneck Transformer Autoencoder

	CIC	Yahoo	Credit card
D_MODEL	64	64	64
NHEAD	4	4	4
NUM_LAYERS	2	2	2
DIM_FF	256	256	256
LATENT_DIM	32	32	32
EPOCHS	50	20	20
BATCH_SIZE	256	256	512

LATENT_DIM=32 gives a uniform 50% compression relative to D_MODEL=64 across all three datasets. **Known issue:** the CIC bottleneck-transformer numbers currently in `results_analysis.md` (§2) were produced before this fix, with LATENT_DIM=64=D_MODEL (i.e. no actual bottleneck) – a re-run with the corrected value is needed before those numbers can be treated as final.

1.5.7 Causal Transformer

	CIC	Yahoo
D_MODEL	64	64
NHEAD	4	4
NUM_LAYERS	2	2
DIM_FF	256	256
DROPOUT	0.1	0.1
EPOCHS	30	30
BATCH_SIZE	256	256

1.6 Evaluation Protocol

1.6.1 Metrics Reported

Every run produces a text report containing the following:

Metric	Notes
Precision / Recall / F1 (per class)	Via <code>sklearn.classification_report</code> , reported for both the normal and anomaly class, plus accuracy and macro/weighted averages.
ROC-AUC	Reported for completeness / comparability with prior literature; not the primary ranking metric under class imbalance.
Average Precision (PR-AUC)	Threshold-independent; the standard choice under severe class imbalance, since ROC-AUC can look deceptively good when negatives vastly outnumber positives.
Balanced Accuracy	$\frac{1}{2}(\text{sensitivity} + \text{specificity})$ at the selected operating threshold.
F1 (positive/attack class)	At the selected operating threshold.
Confusion matrix counts	Raw TP/TN/FP/FN at the selected threshold.

For CIC-IDS2017 specifically, `evaluate_per_class()` additionally reports Average Precision broken down per attack type (treating each attack class against all benign samples in isolation) – this is what supports the per-attack-type discussion in §2.

PR-AUC (Average Precision) is treated as the primary metric for model comparison throughout this thesis, rather than F1 or ROC-AUC. This follows Kim et al. (2022), who observe that pre-defining a decision threshold without access to the test set is impractical, and that existing time-series anomaly detection methods instead report F1 at whichever threshold happens to maximize it – making reported results threshold-dependent by construction. In their Discussion, they conclude that “additional metrics with the reduced dependency such as AUROC or area under precision-recall (AUPR) curve will help in rigorous evaluation”, which is the justification adopted here for ranking models by PR-AUC rather than by threshold-dependent F1.

1.6.2 Threshold Selection

A single decision threshold is still chosen per model/dataset to report the secondary threshold-dependent metrics (F1, balanced accuracy, confusion matrix) discussed above:

it scans all thresholds on the precision-recall curve and picks the one maximizing F_β with $\beta = 2$, weighting recall twice as heavily as precision. The exact value of β is not tuned – any $\beta > 1$ would express the same operationally reasonable preference for an anomaly/fraud-detection setting, where a missed anomaly is typically costlier than a false alarm; $\beta = 2$ was simply fixed as a representative choice.

This is an *oracle* threshold: it is selected using the test set’s own precision-recall curve, which would not be available at deployment time. This is acceptable here because the object of study is relative model/paradigm comparison at the best achievable operating point, not deployment readiness – and because PR-AUC, the primary metric, does not depend on threshold selection at all. The F2/F1/balanced-accuracy figures at the oracle threshold should be read as secondary, illustrative numbers.

1.6.3 No Point Adjustment

Many time-series anomaly detection papers apply *Point Adjustment* (PA): if any point within a contiguous ground-truth anomaly segment is flagged, the entire segment is retroactively counted as detected. Kim et al. (2022) show this can inflate F1 to near-1 even for random scores when anomaly segments are long, and that PA-adjusted F1 is nearly uncorrelated with the un-adjusted F1 (Pearson $r = -0.59$ on the SWaT dataset in their study). No PA is applied anywhere in this project: window/flow labels are fixed at preprocessing time and predictions are scored exactly as thresholded, with no retroactive adjustment.

Chapter 2

Experimental Results

2.1 Credit Card Fraud Detection (Dataset A)

2.1.1 Overall Comparison

Table 2.1 summarises all six evaluated models, ranked by Average Precision (AP), the primary metric. No Causal Transformer variant used for this dataset.

Model	Paradigm	ROC-AUC	Avg. Precision	F1 (fraud)
MLP Autoencoder	Reconstruction	0.9649	0.7105	0.6992
Isolation Forest	Partition	0.9587	0.6725	0.6070
LSTM Autoencoder	Reconstruction (pseudo-seq.)	0.9476	0.6223	0.7362
OC-SVM	Boundary	0.9354	0.5959	0.6454
Transformer Bottleneck AE	Reconstruction (pseudo-seq.)	0.9583	0.5947	0.6137
Predictive LSTM	Predictive (pseudo-seq.)	0.9581	0.5190	0.5244

Table 2.1: Credit Card Fraud Detection – model comparison, ranked by Average Precision.

All six models reach $\text{ROC-AUC} > 0.93$, but AP spans 0.519–0.711 $\rightarrow$ ROC-AUC is far less sensitive to class imbalance than AP and can look deceptively uniform across models that behave very differently once the positive class is rare.

2.1.2 Discussion

No genuine sequence exists in this dataset. Unlike Yahoo S5 or the CIC-IDS2017 temporal/context variants, transaction rows have no temporal or causal ordering across features: **Time** — seconds since the dataset’s first transaction, an arbitrary reference point — is dropped at preprocessing; the pipeline predates the sequential models, added later purely for cross-dataset comparability, so it was left unchanged, leaving each row as an independent 29-dimensional vector (**V1--V28** PCA components + **Amount**). The three "temporal" architectures evaluated here (Predictive LSTM, LSTM AE, Transformer

Bottleneck AE) do not see a real sequence – they reshape each 29-feature row into an artificial sequence of 29 scalar steps $((N, 29) \rightarrow (N, 29, 1))$, one feature per step, in the order present in the .csv file.

Impact on results. Predictive LSTM is the weakest model (AP 0.519): its next-step objective assumes a causal relationship between consecutive steps, but adjacent PCA components are orthogonal/decorrelated by construction, so there is no real next-step signal to learn – reflected in its worst false-positive count (FP=566). LSTM AE holds up comparatively well (AP 0.622, best F1 of all evaluated models at 0.736, lowest FP at 174) - its encoder still compresses the full input into one summary hidden state before decoding, functioning similarly to the MLP AE’s bottleneck rather than depending on order alone. Transformer Bottleneck AE (AP 0.595) sits close to the classical models for the same reason – global average pooling into a single latent vector, does not only rely on positional coherence between adjacent PCA components. MLP Autoencoder, the only architecture that treats the 29 features as an ordinary flat vector with no artificial windowing, wins overall (AP 0.7105) – consistent with there being no sequence to exploit.

This is the mirror image of the CIC-IDS2017 finding that host-context features (encoding *real* temporal/behavioural information) dominate, and of the Yahoo finding that the predictive objective wins under genuine periodic structure. Here, imposing sequence structure where none exists is a handicap: results rank models roughly by how much each architecture depends on the artificial ordering being meaningful – MLP AE (none) > LSTM AE / Transformer Bottleneck (soft, pooled/compressed) > Predictive LSTM (hard, depends on order).

Paradigm-level notes. Reconstruction is the strongest paradigm overall once the pseudo-sequence handicap is accounted for (MLP AE, LSTM AE, and Transformer Bottleneck AE occupy three of the top four spots). Isolation Forest is the best non-reconstruction model (AP 0.6725) and the second-best model overall – notably beating OC-SVM and two of the three temporal architectures, echoing CIC-IDS2017’s finding that simple, low-assumption models remain competitive whenever imposed structure is a poor match for the data. OC-SVM (AP 0.5959) sits mid-table, with the second-highest false-positive count after Predictive LSTM. Predictive LSTM is the only representative of the predictive paradigm on this dataset and the weakest model overall, for the structural reason discussed above. Given this, a Causal Transformer variant was not trained for this dataset — there is no genuine sequence for any predictive architecture to exploit.

Sanity check. LSTM AE’s best-of-all F1 (0.7362) despite a middling AP (0.6223) is a reminder that F1 is computed at a single oracle F2-threshold and can favor a different operating point than the one maximising area under the PR curve; AP remains the

primary metric for cross-model ranking.

2.2 Yahoo Webscope S5 (Dataset B)

A2 (single clean periodic component + trend, point outliers) is excluded from the comparisons below: like A1, it contains only point anomalies rather than contextual or level-shift ones, so it would pose the same limited-discriminative-power challenge discussed for A1 without adding a new anomaly type to the comparison.

2.2.1 A1 – Point Anomalies (Real Server Metrics)

Table 2.2 summarises the five models evaluated on A1. Causal Transformer and Transformer Bottleneck AE were not run on this benchmark.

Model	Paradigm	ROC-AUC	Avg. Precision
MLP Autoencoder	Reconstruction	0.792	0.611
LSTM Autoencoder	Reconstruction	0.794	0.600
OC-SVM	Boundary	0.776	0.595
Predictive LSTM	Predictive	0.789	0.588
Isolation Forest	Partition	0.765	0.511

Table 2.2: Yahoo S5 A1 (point anomalies) – model comparison, ranked by Average Precision.

Discussion. All five models cluster tightly within AP 0.51–0.61: MLP Autoencoder is best (AP 0.611) and Isolation Forest worst (AP 0.511), but the gap is narrow enough that no architecture shows a structural advantage here. Point anomalies (single-timestep spikes) do not require temporal understanding to detect. The window-level label blur discussed in §1.4 (a 1.76% point-level anomaly rate becomes ~ 17 – 18% at the window level) further limits all models roughly equally, since it dilutes the discriminative signal available to every architecture in the same way.

2.2.2 A3 – Contextual/Seasonal Anomalies

Table 2.3 summarises all seven models evaluated on A3.

Discussion. Results diverge sharply once anomalies become contextual (a value normal in absolute terms but wrong for its phase in the seasonal cycle) rather than amplitude-based. Isolation Forest and OC-SVM collapse toward chance (ROC-AUC 0.511 and 0.584): a flattened 64-step window carries no phase information, so distribution-based detection fails outright. LSTM Autoencoder collapses just as badly (ROC-AUC 0.566) despite

Model	Paradigm	ROC-AUC	Avg. Precision
Predictive LSTM	Predictive	0.868	0.812
Transformer Bottleneck AE	Reconstruction	0.762	0.663
Causal Transformer	Predictive	0.738	0.623
MLP Autoencoder	Reconstruction	0.765	0.507
LSTM Autoencoder	Reconstruction	0.566	0.392
OC-SVM	Boundary	0.584	0.376
Isolation Forest	Partition	0.511	0.331

Table 2.3: Yahoo S5 A3 (contextual/seasonal anomalies) – model comparison, ranked by Average Precision.

being a sequence model – its encoder compresses the whole seasonal window into a single hidden state, showing this is a compression-bottleneck failure rather than a weakness of recurrent architectures generally. In the same vein, Transformer Bottleneck AE trails the best model (AP 0.663) but is by far the strongest *reconstruction*-based architecture, well ahead of LSTM AE: global self-attention over the window likely recovers periodicity that the LSTM AE’s bottleneck destroys, which probably accounts for the gap. Predictive LSTM dominates and is the best model on any Yahoo benchmark (AP 0.812, ROC-AUC 0.868): its next-step objective directly exploits how predictable a seasonal series is, so an injected anomaly produces a clean spike in prediction error exactly where it violates the expected trajectory.

2.2.3 A4 – Level-Shift Anomalies

Table 2.4 summarises all seven models evaluated on A4.

Model	Paradigm	ROC-AUC	Avg. Precision
Predictive LSTM	Predictive	0.745	0.505
Causal Transformer	Predictive	0.695	0.436
MLP Autoencoder	Reconstruction	0.719	0.402
Transformer Bottleneck AE	Reconstruction	0.655	0.404
Isolation Forest	Partition	0.498	0.268
OC-SVM	Boundary	0.519	0.265
LSTM Autoencoder	Reconstruction	0.502	0.252

Table 2.4: Yahoo S5 A4 (level-shift anomalies) – model comparison, ranked by Average Precision.

Discussion. The series now model a system that experiences abrupt structural shifts. Every model degrades relative to A3, regardless of paradigm: windows from all ~ 100 series are pooled together without series identity, and each series sits at its own baseline level – a

window’s absolute value alone cannot tell a model whether it is looking at a genuine post-shift window or an ordinary window from another series whose normal baseline happens to be similar. Classical models bottom out entirely under this confound – Isolation Forest reaches ROC-AUC 0.498, effectively a coin flip, since a shifted window can fall in a region of the pooled feature space already dense with unrelated series’ normal windows. Reconstruction autoencoders (MLP AE, Transformer Bottleneck AE) are blunted for the same reason and fail to overtake Predictive LSTM here despite a step-shaped window intuitively being easier to flag by reconstruction error; both plateau around AP 0.40. LSTM Autoencoder additionally collapses outright (ROC-AUC 0.502), reproducing the same encoder-compression failure seen on A3. Predictive LSTM still leads (AP 0.505, ROC-AUC 0.745), though its margin over the field narrows sharply from A3: because it conditions each prediction on a series’ own recent history rather than the pooled baseline distribution, a shift still registers as a violation of local continuity regardless of the absolute level it occurs at, but it probably generates a sustained, noisier error signal over the transition rather than the clean seasonal-deviation spike it could more easily exploit on A3.

2.2.4 Trivial Baseline Sanity Check (Yahoo Only)

Following Kim et al. (2022), who recommend checking every trained model against at least one label-free baseline before attributing its score to genuine representation learning, the models trained on Yahoo S5 were also evaluated against such a baseline (their ”Case 2”): score each window by its raw L2 norm $\|w\|_2$, with no learned model at all. Because point anomalies in A1–A3 are amplitude spikes, a window containing one has elevated magnitude even without any representation learning. Any trained model that fails to exceed this baseline on PR-AUC is not adding value over raw signal amplitude – this check is what confirms that the gains seen on A3 are genuine and not a window-magnitude artifact.

On A3, the baseline itself scores ROC-AUC 0.508 and AP 0.348 – indistinguishable from random, as expected for contextual anomalies that are phase violations rather than amplitude spikes. The *margin* each trained model holds above this baseline is a more informative quantity than its raw AP: the baseline’s own AP (0.348) closely matches the $\sim 33\%$ positive-window prior on A3 (itself elevated by the window-level label blur discussed in §1.4), confirming the baseline is pure noise and that a model’s raw AP is partly a function of this inflated prior rather than of learned structure alone.

- **LSTM AE (+0.04 AP):** barely above noise – encoder compression discards phase, leaving the model effectively using magnitude alone.
- **MLP AE (+0.16 AP):** captures some window-shape statistics implicitly.

- **Transformers (+0.28–0.29 AP):** global attention over the 64-step window recovers periodicity directly, a large and meaningful margin.
- **Predictive LSTM (+0.47 AP):** more than doubles the baseline’s AP – the strongest evidence in this project that the *training objective* (predict the next step), not raw model capacity, is the decisive factor on contextual anomalies.

2.2.5 Discussion

Paradigm and architecture both matter. The A3 results form a 2×2 comparison of paradigm (reconstruction vs. predictive) against architecture (LSTM vs. Transformer): reconstruction scores 0.39 AP (LSTM AE) and 0.66 AP (Transformer Bottleneck), while predictive scores 0.81 AP (Predictive LSTM) and 0.62 AP (Causal Transformer). The predictive paradigm is necessary – both predictive models beat both reconstruction models – but not sufficient on its own: the 0.19 AP gap between Predictive LSTM and Causal Transformer, despite an identical training objective and identical training data, shows the LSTM architecture carries an additional advantage on seasonal data. The likely explanation is that the LSTM’s recurrent hidden state acts as a continuous phase tracker, accumulating a compact summary of where the signal sits in its seasonal cycle at every step, so a phase violation produces a clean error spike; the causal transformer can attend to any earlier position directly but must learn explicit attention patterns to recover phase, a harder problem for A3’s three interacting frequency components. On A4 this architectural gap shrinks to 0.07 AP, since a single abrupt transition does not require phase accumulation, so the LSTM’s sequential inductive bias matters less.

Per-anomaly-type summary. Point anomalies (A1) leave all models within a narrow 0.51–0.61 AP band – no temporal understanding is needed to flag an isolated spike. Seasonal anomalies (A3) are where the predictive paradigm’s advantage is clearest, and where architecture choice matters most within that paradigm. Change-points (A4) degrade every model, but Predictive LSTM still leads, and Causal Transformer still beats both reconstruction models – the predictive advantage shrinks with weaker, shorter-lived signal, but does not flip. Classical models (Isolation Forest, OC-SVM) and LSTM Autoencoder collapse on any anomaly type that requires contextual rather than absolute discrimination (A3, A4), while remaining competitive only on A1.

2.3 CIC-IDS2017 (Dataset C)

2.3.1 Overall Comparison

Before comparing models, Table 2.5 breaks down the pooled attack flows in the flat/context test set by class, since every AP figure reported below is computed against this entire pool at once and not per class. This includes **DoS Hulk** (158,469 flows, tied with PortScan for the largest attack class): every preprocessing script labels any non-benign, non-attempted flow as positive with no per-class filtering, so DoS Hulk flows are counted as attack samples in every model’s aggregate ROC-AUC/AP/F1 in Table 2.7. However, Engelen et al. (WTMC 2021) show the DoS Hulk attack tool used to generate this dataset is misconfigured – it sets the HTTP `Connection` header to `Close` instead of `Keep-Alive`, so the target web server terminates the connection immediately instead of being kept busy, rendering the attack ineffective by construction. Consequently, DoS Hulk is deliberately excluded from the *per-class* breakdown in §2.3.2. This exclusion applies only to the per-class reporting, not to the pooled evaluation: DoS Hulk’s $\sim 158\text{k}$ flows remain part of the positive class that every aggregate metric in Table 2.7 is computed against.

Attack class	n (flows)	% of attack pool
PortScan	159,000	36.66%
DoS Hulk	158,469	36.54%
DDoS	95,000	21.91%
DoS GoldenEye	7,500	1.73%
DoS slowloris	4,000	0.92%
FTP-Patator	4,000	0.92%
SSH-Patator	3,000	0.69%
DoS Slowhttptest	1,700	0.39%
Bot	738	0.17%
Web Attack – Brute Force	151	0.03%
Infiltration	32	0.01%
Web Attack – XSS	27	0.01%
Web Attack – SQL Injection	12	<0.01%
Heartbleed	11	<0.01%

Table 2.5: CIC-IDS2017 – attack-class composition of the pooled attack flows in the flat/context test set. DoS Hulk contributes to every aggregate metric in Table 2.7 but is excluded from the per-class breakdown in §2.3.2 (see text).

PortScan, DoS Hulk, and DDoS jointly account for 95.1% of all attack flows in this pool – directly substantiating the claim, made repeatedly below, that aggregate AP on this dataset is effectively a proxy for performance on these three classes, and that the ten remaining classes (all under 2% individually, seven of them under 0.2%) have limited leverage over the model ranking in Table 2.7 regardless of how well or poorly a model handles them individually. This is also why a per-attack breakdown is necessary at all

(§2.3.2 below): a model could in principle top Table 2.7 while being essentially blind to every interpretable attack type except PortScan and DDoS.

Table 2.6 gives the analogous breakdown for the temporal pipeline’s windowed test set. DoS Hulk is included here for the same reason as above.

Attack class	n (windows)	% of attack pool
PortScan	159,160	36.42%
DoS Hulk	158,469	36.27%
DDoS	95,131	21.77%
DoS GoldenEye	7,575	1.73%
DoS slowloris	4,001	0.92%
FTP-Patator	3,966	0.91%
Bot	3,320	0.76%
SSH-Patator	2,980	0.68%
DoS Slowhttptest	1,742	0.40%
Infiltration	445	0.10%
Web Attack – Brute Force	151	0.03%
Web Attack – XSS	26	0.01%
Web Attack – SQL Injection	20	<0.01%
Heartbleed	19	<0.01%
BENIGN	1,282,611	–

Table 2.6: CIC-IDS2017 – attack-class composition of the pooled attack windows in the temporal test set (1,719,616 windows total, 25.41% attack rate).

Window counts track flat flow counts closely for dense, bursty classes (DoS Hulk: 158,469 vs. 158,469 flat; similarly PortScan, DDoS, the DoS/Patator classes) – sliding `STRIDE_TEST=1` across a long uninterrupted attack span relabels nearly every position as attack without adding windows beyond what the span already spans. Sparse, isolated classes inflate far more sharply – Bot $\sim 4.5\times$ ($738 \rightarrow 3,320$), Infiltration $\sim 14\times$ ($32 \rightarrow 445$) – since each isolated flow gets ”smeared” across up to `WINDOW=16` overlapping windows under the any-attack rule (§1.4). Table 2.7 summarises all thirteen model/feature-set combinations evaluated on CIC-IDS2017, ranked by Average Precision (AP), the primary metric. Attack prevalence in the test set is $\sim 25\%$, considerably less extreme than Credit Card’s 0.17% but still imbalanced enough that AP and ROC-AUC diverge for the weaker models.

Model	Feature set	Paradigm	ROC-AUC	Avg. Precision	Balanced Acc.	F1 (attack)
MLP Autoencoder	Context	Reconstruction	0.9860	0.9487	0.9548	0.9020
OC-SVM	Context	Boundary	0.9745	0.9396	0.9569	0.9229
Isolation Forest	Context	Partition	0.9785	0.9328	0.9270	0.8367
LSTM Autoencoder	Temporal	Reconstruction	0.8976	0.7955	0.7846	0.6249
MLP Autoencoder	Flat	Reconstruction	0.9189	0.7921	0.8022	0.6305
Isolation Forest	Flat	Partition	0.9049	0.7612	0.8228	0.6639
OC-SVM	Temporal	Boundary	0.8629	0.7816	0.7600	0.5888
MLP Autoencoder	Temporal	Reconstruction	0.8831	0.7509	0.7734	0.6125
Isolation Forest	Temporal	Partition	0.8438	0.7604	0.6841	0.5195
OC-SVM	Flat	Boundary	0.7336	0.6845	0.6121	0.4644
Bottleneck Transformer AE	Temporal	Reconstruction	0.8077	0.6539	0.7059	0.5423
Causal Transformer	Temporal	Predictive	0.7099	0.5449	0.5520	0.4320
Predictive LSTM	Temporal	Predictive	0.6126	0.4472	0.5475	0.4295

Table 2.7: CIC-IDS2017 – all model/feature-set combinations, ranked by Average Precision.

Host-context features dominate. The three context-augmented models occupy the top three positions by every metric, with AP ranging 0.933–0.949. The gap between the best context model (AE+Context, AP 0.949) and the best non-context model (Flat MLP AE, AP 0.792) exceeds the entire spread across the remaining ten models. Each of the three base architectures improves substantially when context features are added: IForest gains +0.172 AP (0.761 $\rightarrow$ 0.933), MLP AE gains +0.157 (0.792 $\rightarrow$ 0.949), and OC-SVM gains the most, +0.255 (0.685 $\rightarrow$ 0.940) – the model that was worst among the flat trio becomes roughly tied for best. This shows that better features are sometimes more important than the choice of paradigm: OC-SVM’s RBF kernel is the same kernel finding the same kind of hypersphere boundary either way, but the 9 added spatial dimensions give it a well-separated kernel problem where the flat 77-feature space previously left benign and PortScan regions heavily overlapping.

Temporal windowing alone does not help. For IForest and the MLP AE, grouping flows into src-IP-ordered windows (§1.4) without adding context features gives no consistent improvement over the flat view, and sometimes hurts: Flat IForest (AP 0.761) $\approx$ Temporal IForest (AP 0.760), while Flat AE (AP 0.792) $>$ Temporal AE (AP 0.751). This is a direct contrast with the host-context gain above: both modifications add information about flow *order/activity over time*, but only the aggregate context statistics translate into a usable decision signal for the two attack classes (DDoS, PortScan) that dominate the test set by volume. The temporal windows carry the same underlying activity information implicitly, spread across 16 arbitrarily-ordered flows, which is a harder representation for these architectures to exploit directly.

Reconstruction beats prediction. Among the temporal-feature models, reconstruction-based architectures (LSTM AE, Temporal AE, Temporal OC-SVM) all clearly outperform the two prediction-based ones (Predictive LSTM AP 0.447, Causal Transformer AP 0.545) – the reverse of the Yahoo A3 finding (§2). Both predictive models’ threshold sweeps land at an extreme operating point (FP > 1.1M out of 1.28M benign flows), consistent with their low ROC-AUC (0.61 and 0.71): the scores themselves are poorly discriminative, not merely mis-thresholded.

2.3.2 Per-Attack-Type Breakdown

Table 2.8 breaks down Average Precision by attack class for the three context models, the strongest family overall. Class sizes vary by four orders of magnitude (11 to 159,000 samples), which itself shapes what is detectable at all.

Attack class	n	IForest+Ctx	OC-SVM+Ctx	AE+Ctx
DDoS	95,000	0.982	0.998	0.994
PortScan	159,000	0.601	0.820	0.842
DoS GoldenEye	7,500	0.655	0.215	0.220
DoS slowloris	4,000	0.190	0.012	0.025
DoS Slowhttptest	1,700	0.112	0.007	0.013
Heartbleed	11	0.093	0.001	0.000
Infiltration	32	AP 0.03–0.14, no clear winner across any model		
Bot	738	<0.01 across all models, including context variants		
FTP-Patator	4,000	<0.01 across all models, including context variants		
SSH-Patator	3,000	<0.01 across all models, including context variants		
Web Attack subtypes	12–151	near-zero AP everywhere (Brute Force, SQL Injection, XSS)		

Table 2.8: CIC-IDS2017 – per-attack-class Average Precision for the three context-augmented models.

Consistently well-detected. Two attack classes stand out as reliably detectable across nearly every feature set:

- **DDoS** is the easiest class by a wide margin: all context models score AP > 0.98 (OC-SVM+Context reaches 0.998), and even the flat models exceed AP 0.77. The very high packet rate and tiny packet sizes characteristic of a flood produce an extreme outlier in both per-flow and host-context feature spaces – `ctx_max_flow_cnt` alone nearly isolates it.
- **PortScan** is the clearest demonstration of the context features’ value: every flat and temporal model scores AP 0.07–0.28, while every context model exceeds 0.60, with AE+Context reaching 0.842. A single short scan flow to an arbitrary port is unremarkable in isolation, but `ctx_max_port_cnt` (unique destination ports contacted by the source IP in the preceding 60 seconds) is unmistakable during a scan

- probably helping especially with this attack. IForest+Context is comparatively weaker here (0.601).

Mixed context response for application-layer DoS. Not every attack class responds to context features the same way – the three application-layer DoS classes split the context family rather than favouring it uniformly:

- **DoS GoldenEye** shows a clear split within the context family: IForest+Context reaches AP 0.655, while AE+Context and OC-SVM+Context both collapse to ~ 0.22 . GoldenEye floods a single HTTP endpoint, producing many flows to one destination port – high `ctx_max_flow_cnt` paired with very low `ctx_max_port_rate`. Isolation Forest isolates this specific combination directly; the autoencoder reconstructs it with moderate error spread across many feature dimensions, reducing its discriminative power for this attack specifically.
- **DoS Slowhttptest and DoS slowloris** show the same pattern, more muted: IForest+Context is the only context model with meaningful detection (AP 0.112 and 0.190), though it still falls far short of Temporal IForest’s AP 0.69/0.61 on the same classes. Slow HTTP attacks maintain many concurrent long-duration connections to a single port – the resulting context signature (elevated `ctx_max_flow_cnt`, near-zero `ctx_max_port_rate`) is a tight, isolated cluster that IForest’s isolation mechanism targets well, while AE+Context and OC-SVM+Context fail to exploit it, scoring near-zero. The 60-second context window is a partial mismatch here, since Slowloris connections last minutes and many flows fall outside it – Temporal IForest’s explicit per-host chronological ordering still provides the best detection for this attack family.

Largely undetectable regardless of feature set. FTP-Patator, SSH-Patator, Bot, and the three Web Attack subtypes all score $AP < 0.01$ across every model, context included. Credential brute-force over legitimate protocols and HTTP-level attacks (SQL injection, XSS) produce flow statistics and host-context activity indistinguishable from normal usage – detecting them would require failed-authentication counters or payload inspection, neither available in a flow-statistics feature set. Heartbleed (11 samples) and Infiltration (32 samples) are too small to draw strong conclusions from, though IForest+Context’s modest edge on Heartbleed (AP 0.093 vs. near-zero for the other two context models) is consistent with the same axis-aligned-isolation advantage seen on the slow-DoS classes.

2.3.3 Discussion

Host-context features are the single most impactful addition in this study. Appending 9 rolling-window per-host activity statistics (§1.4) raises AP from 0.79 to 0.95 across the board – a larger gain than any other modelling choice in this project, larger than the choice of paradigm, and larger than moving from a flat to a temporal view of the same data – driven primarily by solving the PortScan blind spot that every flat and temporal model shared. No single context model dominates every attack type, even though AE+Context wins on the aggregate AP that is driven by the two largest classes.

A practical ensemble. No single model or feature set covers every attack class on this dataset. Pairing AE+Context (strong on DDoS and PortScan, the two classes that dominate the test set by volume) with Temporal IForest (strong on the slow-DoS classes and, tentatively, Heartbleed) would cover the widest range of detectable attack classes within this feature set – a concrete illustration of why the per-attack breakdown in §2.3.2 matters more than any single aggregate ranking. This premise, and a combined-score ensemble built and evaluated against it, are tested directly in §3.3: the two models’ evidence is confirmed genuinely disjoint, but combining their scores turns out to trade PortScan detection quality for gains on the slow-DoS classes rather than covering both for free.

2.4 Exploratory Data Analysis

This section collects the exploratory plots underlying the dataset-specific claims made in §1.4 and throughout this chapter.

2.4.1 Credit Card Fraud Detection

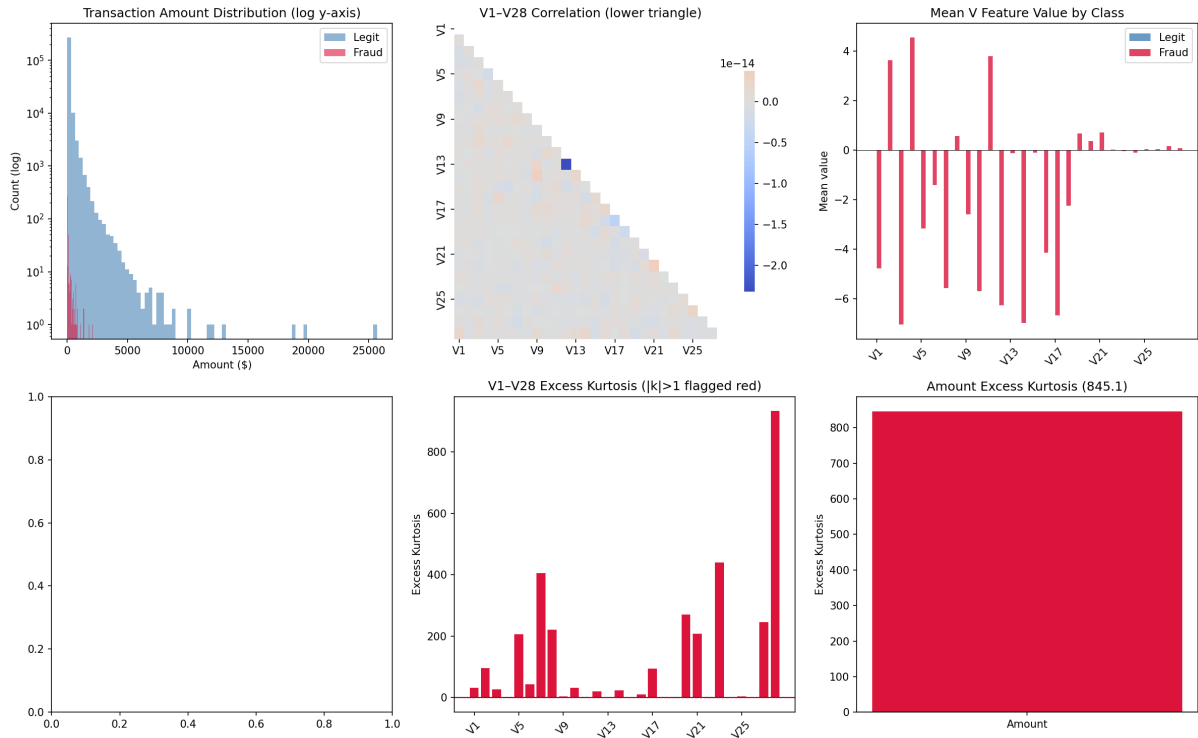


Figure 2.1: Credit Card Fraud Detection – transaction amount distribution, V1–V28 correlation structure, mean V-feature value by class, and excess kurtosis per feature (flagging the heavy-tailed PCA components discussed in the Datasets section).

2.4.2 Yahoo Webscope S5

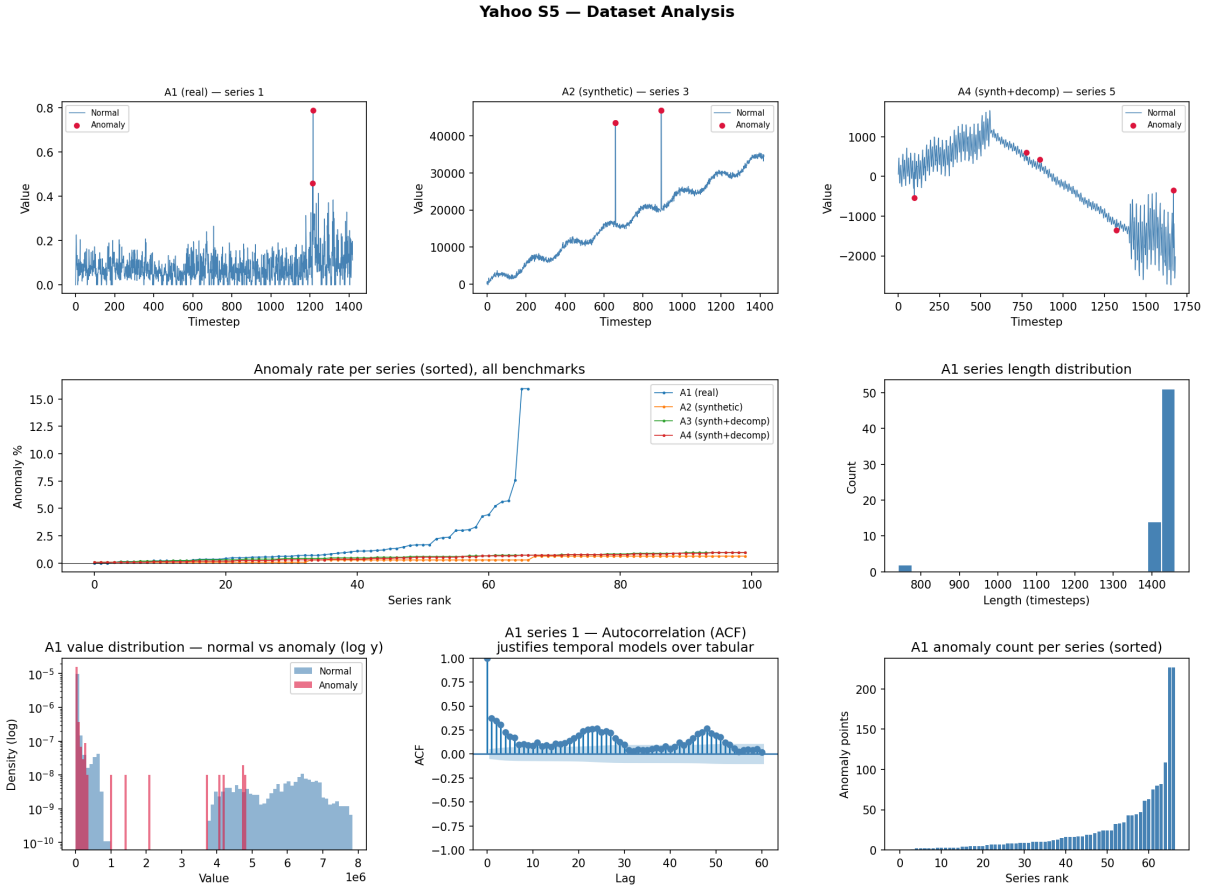


Figure 2.2: Yahoo S5 — example series per benchmark, per-series anomaly rate, A1 series length distribution, A1 value distribution for normal vs. anomalous points, A1 autocorrelation (motivating the use of temporal models over tabular ones), and A1 anomaly count per series.

2.4.3 CIC-IDS2017

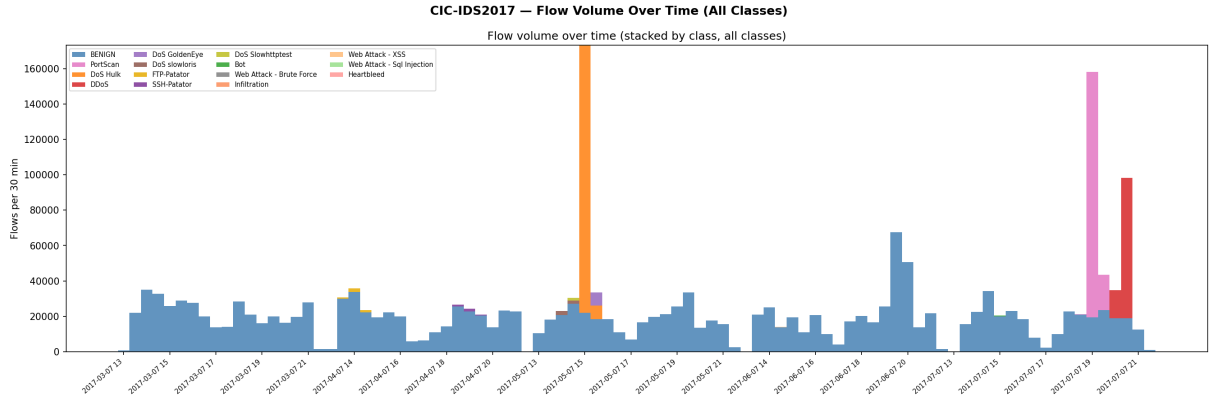


Figure 2.3: CIC-IDS2017 – flow volume over the capture week, stacked by class. BENIGN traffic dominates volume outside the concentrated attack windows visible on Tuesday–Friday.

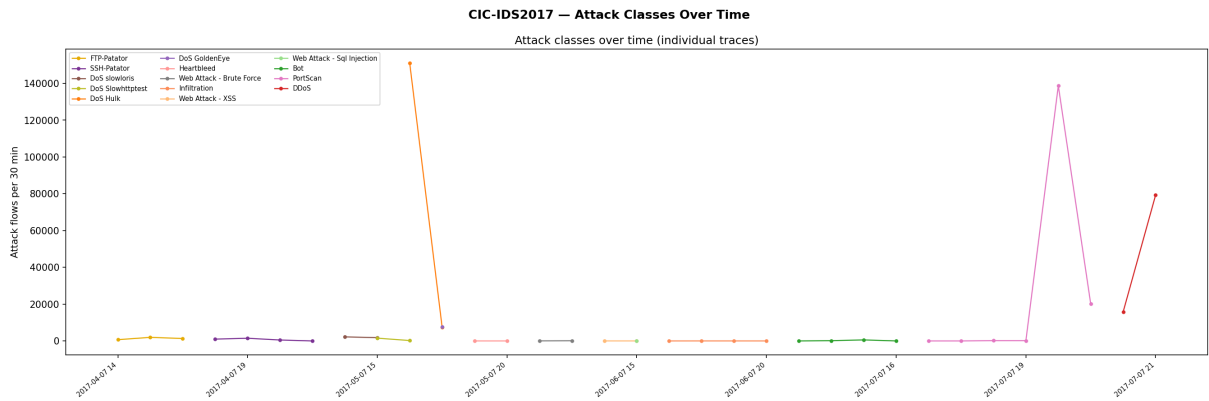


Figure 2.4: CIC-IDS2017 – attack classes over time, individual traces. Each attack occupies a short, distinct time window, confirming that attack classes do not overlap temporally within the capture.

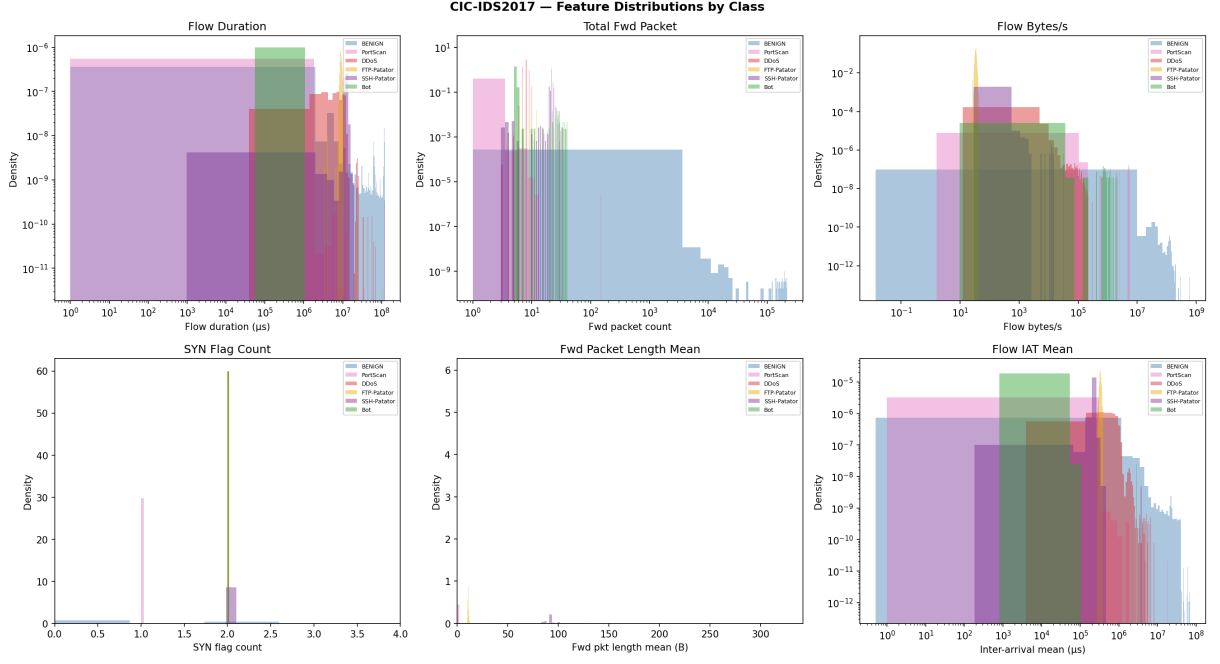


Figure 2.5: CIC-IDS2017 – per-class distributions of six flow-level features (flow duration, forward packet count, flow bytes/s, SYN flag count, forward packet length mean, inter-arrival mean) for BENIGN and five representative attack classes.

2.5 Explainability

This section is a placeholder for the explainability analysis (SHAP attributions, reconstruction-error heatmaps, latent-space visualisation, and related diagnostics), to be filled in as that work is completed.

Chapter 3

Explainability Case Studies

This chapter tests specific mechanistic claims made in Chapter 2 with targeted diagnostics, rather than relying on the aggregate metrics alone to support the mechanisms offered there.

3.1 Reconstruction-Error Localization on Yahoo A3

The A3 discussion in Chapter 2 attributes the Transformer Bottleneck Autoencoder’s lead over the LSTM Autoencoder (AP 0.663 vs. 0.392, ROC-AUC 0.813 vs. 0.566) to a mechanistic difference between the two bottlenecks: the LSTM Autoencoder compresses an entire 64-step window into a single final hidden state, which is hypothesized to destroy the window’s seasonal phase information and produce reconstruction error that is smeared diffusely across all timesteps; the Transformer’s global self-attention, by contrast, is hypothesized to preserve enough positional information to localize its reconstruction error sharply at the specific timestep where the window’s phase is violated. This is a plausible mechanism, but as stated it is inferred from the aggregate metrics alone – neither model’s per-timestep error was actually inspected. This section tests it directly.

3.1.1 Initial Assumption

The hypothesis under test makes an explicit, checkable prediction: for anomalous A3 windows, the LSTM Autoencoder’s per-timestep squared reconstruction error should look approximately *uniform* across the window’s 64 positions, while the Transformer’s per-timestep error should show a sharp, localized peak at the position where the seasonal pattern is violated. If instead both models produce similarly shaped error profiles, the AP/ROC-AUC gap would need a different explanation than the one offered.

3.1.2 Qualitative Check: Per-Timestep Error Heatmaps

Both trained models were reconstructed on every anomalous A3 test window, and the per-timestep squared error was plotted as a heatmap (one row per window, one column per of the 64 timesteps), with a random subsample of 300 windows used to keep the rendered image legible. Figures 3.1 and 3.2 show the result.

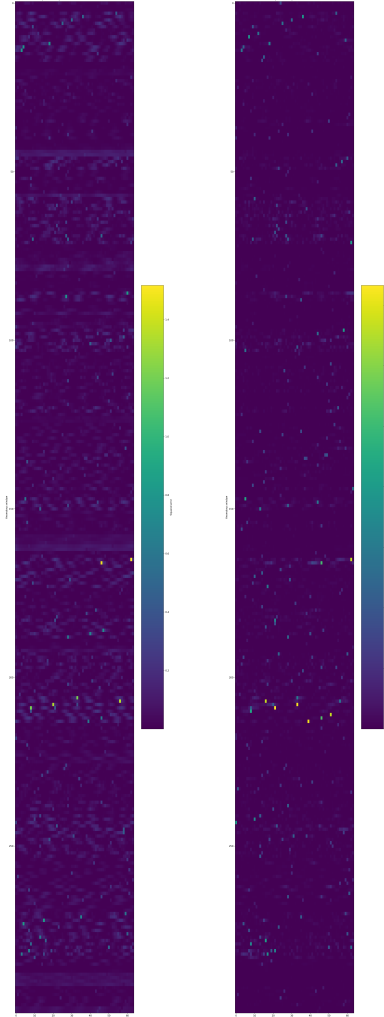


Figure 3.1: LSTM Autoencoder – per-timestep squared reconstruction error on 300 randomly sampled anomalous A3 windows (one row per window, left). Elevated error appears as sparse, punctate points rather than as a uniform band, but a visibly larger share of rows carry a low-level elevated baseline across most of their width compared to the Transformer.

Figure 3.2: Transformer Bottleneck Autoencoder – per-timestep squared reconstruction error on the same 300 anomalous A3 windows, same row order and colour scale convention as Figure 3.1 (right). Error is visibly more sparse and confined to isolated timesteps.

The visual comparison is broadly consistent with the initial assumption – the Trans-

former’s error concentrates into fewer, brighter points, while the LSTM Autoencoder’s error is more often spread thinly across a window’s full width – but a heatmap read by eye is a weak instrument here: each panel is normalized to its own colour scale, and the LSTM Autoencoder’s error values are roughly double the Transformer’s in magnitude throughout, which alone would make its background look more saturated regardless of how concentrated the error actually is. A qualitative read of Figures 3.1–3.2 therefore *suggests* the phase-destruction mechanism but cannot confirm it.

3.1.3 Quantitative Verification: Gini Coefficient of Error Concentration

To separate genuine error concentration from an artifact of differing overall error magnitude, each anomalous window’s 64 per-timestep squared errors were summarized with a Gini coefficient – a standard inequality measure (originally used for income distributions) that is invariant to the absolute scale of its input. For a window’s 64 error values sorted ascending, a Gini of 0 means the error is spread perfectly evenly across every timestep, and a Gini of 1 means it is concentrated entirely at a single timestep; unlike a raw magnitude comparison, this metric isolates the *shape* of the error distribution within a window from its overall size.

For n per-timestep squared errors $x_1 \leq x_2 \leq \dots \leq x_n$ sorted ascending (here $n = 64$), the Gini coefficient is computed as

$$G = \frac{2 \sum_{i=1}^n i x_i}{n \sum_{i=1}^n x_i} - \frac{n+1}{n}. \quad (3.1)$$

Computed over the full population of 14,449 anomalous A3 windows, the two models separate clearly:

	Mean Gini	Median Gini
LSTM Autoencoder	0.566	0.606
Transformer Bottleneck Autoencoder	0.721	0.724

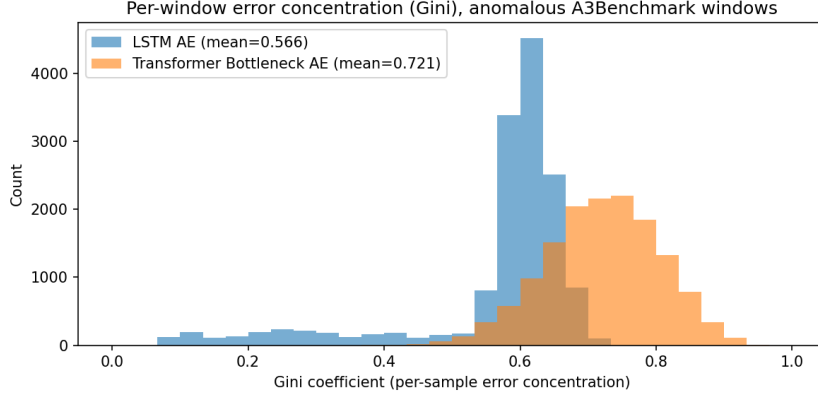


Figure 3.3: Distribution of per-window Gini coefficients across all 14,449 anomalous A3 windows. The Transformer’s distribution is unimodal and shifted toward high concentration (Gini > 0.6); the LSTM Autoencoder’s distribution is bimodal, with a majority clustered near the Transformer’s range but a distinct secondary population spread across low Gini values (error genuinely smeared across the window).

This confirms the initial assumption on average: the Transformer’s error is more concentrated than the LSTM Autoencoder’s across the full anomalous population, not merely in the handful of examples a heatmap happens to display. But the distribution in Figure 3.3 reveals a detail the aggregate means alone would miss – the LSTM Autoencoder’s Gini values are *bimodal*, not uniformly shifted down. A majority of its windows cluster in essentially the same high-concentration range as the Transformer’s (Gini ≈ 0.6 , comparable localization ability), while a distinct secondary population of windows sits at low Gini (< 0.4), where the error genuinely is smeared close to evenly across the full window. This indicates the phase-destruction mechanism is not a uniform property of every LSTM Autoencoder reconstruction, but a failure mode affecting a specific subset of windows – and it is this subpopulation, not a general degradation across all anomalous windows, that is responsible for dragging the LSTM Autoencoder’s aggregate AP and ROC-AUC below the Transformer’s, since on the remaining majority of windows the two models localize error comparably well.

3.2 Pseudo-Sequence Reconstruction Error on Credit Card Fraud

The Credit Card Discussion in Chapter 2 argues that the three “temporal” architectures (Predictive LSTM, LSTM Autoencoder, Transformer Bottleneck Autoencoder) do not see a genuine sequence: each reshapes a row’s 29 independent, decorrelated PCA features into an artificial sequence of 29 scalar steps, in whatever order the columns happen to appear in the source CSV. Predictive LSTM’s poor performance (AP 0.519) is attributed

directly to this – “adjacent PCA components are orthogonal/decorrelated by construction, so there is no real next-step signal to learn.” This section tests the reconstruction-based half of that claim (LSTM Autoencoder, Transformer Bottleneck Autoencoder) with the same per-position error diagnostic used in §3.1.

3.2.1 Initial Assumption

If the 29 column positions genuinely carry no meaningful order, per-position reconstruction error on fraud samples should look like unstructured noise: no column should consistently accumulate more error than any other across the fraud population, for either model. A structured, repeated pattern – specific columns lighting up across many fraud samples – would complicate that picture and call for a follow-up explanation.

3.2.2 Qualitative Check: Per-Feature-Position Error Heatmaps

Both trained models were reconstructed on the full fraud test population (492 samples), and the per-column squared error was plotted as a heatmap (one row per sample, one column per of the 29 feature positions), with a random subsample of 300 fraud samples used to keep the rendered image legible. Figures 3.4 and 3.5 show the result.

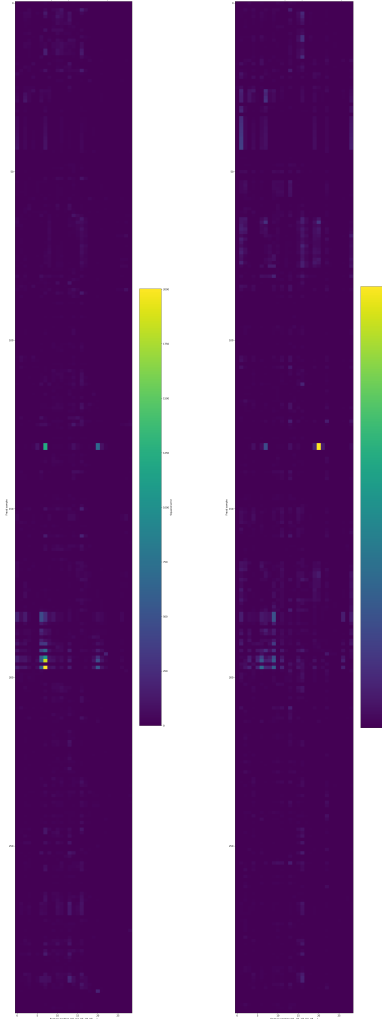


Figure 3.4: LSTM Autoencoder – per-feature-position squared reconstruction error on 300 randomly sampled fraud transactions (one row per sample, columns V1–V28 then Amount, left).

Figure 3.5: Transformer Bottleneck Autoencoder – per-feature-position squared reconstruction error on the same 300 fraud transactions, same row order and column convention as Figure 3.4 (right).

Contrary to the initial assumption, both heatmaps show a visibly structured, repeated pattern: a handful of column positions accumulate elevated error across many fraud rows, rather than error being scattered evenly across all 29 positions. This does not by itself contradict the "no real sequence" claim – it does not show that the models exploit *order*, only that error is not uniformly distributed across *columns*. Two distinct explanations are consistent with a structured heatmap: either specific PCA features are simply harder to reconstruct on fraud rows regardless of where they sit in the row (a feature-*identity* effect, fully compatible with "no real sequence"), or the models are exploiting some incidental adjacency between specific neighbouring columns in this one fixed ordering (a

position effect, which would nuance but not overturn the claim, since that adjacency is still arbitrary and non-causal). The heatmap alone cannot distinguish these.

3.2.3 Quantitative Verification: Column-Shuffle Feature-Identity Check

To separate the two explanations, the same 492 fraud rows were re-run through both models with a fixed random permutation of the 29 columns applied first (seed 42), and the per-column error was re-ranked. If the same underlying features dominate the top-error positions in both the original and the shuffled column order, the effect is feature-identity-driven; if overall error also changes substantially after shuffling, position/adjacency plays a role too.

	Overlap (top 10, orig. vs. shuffled)	Top-error magnitude, orig. $\rightarrow$ shuffled	Stable top features (in both orderings)
LSTM Autoencoder	6/10	47.7 $\rightarrow$ 71.7	V1, V3, V7, V8, V14, V17
Transformer Bottleneck AE	6/10	12.9 $\rightarrow$ 136.8	V2, V3, V7, V8, V14, V17

Both effects are present. Six of the top 10 highest-error features are the same before and after shuffling, for both models – well above the ~ 3 –4 expected in common between two independent random top-10 draws out of 29 features under a pure-noise null. **V14 and V17 are stable top accumulators for both models under both orderings**, and both are independently singled out by the community exploratory notebook cited in Chapter 1 as its two strongest negative correlates with the fraud label – an external check that these two features genuinely carry fraud-relevant signal, not an artifact of this experiment. At the same time, overall error rose substantially after shuffling for both models (LSTM Autoencoder’s top value: 47.7 $\rightarrow$ 71.7; Transformer’s: 12.9 $\rightarrow$ 136.8), which only makes sense if both encoders were also leaning on some incidental adjacency between specific neighbouring columns in the fixed, original CSV order.

Taken together, this confirms the core claim with a refinement the aggregate metrics could not surface on their own: the reconstruction-based architectures did learn to depend on the one fixed, arbitrary column ordering supplied at preprocessing time, beyond what a pure feature-identity effect would predict. The fact that error went up means the models had implicitly learned something that depended on which value sat next to which in the specific column arrangement the CSV happens to use, even though that adjacency is meaningless (it’s just PCA-component arbitrary order) - not evidence of a hidden causal sequence in the data, but more like the models overfit to an arbitrary but fixed layout

3.3 Disjoint Feature Regimes: AE+Context vs. Temporal IForest

The CIC-IDS2017 Discussion in Chapter 2 proposes pairing AE+Context (strong on DDoS and PortScan) with Temporal IForest (strong on the slow-DoS classes and, tentatively, Heartbleed) into a practical ensemble, since the two operate on disjoint feature regimes – host-context features versus a 16-flow temporal window. That argument rests on two separate premises, one per model: that Temporal IForest’s score is genuinely informed by the window’s temporal structure rather than by one flow’s static feature values sitting inside a wider, functionally inert vector; and that AE+Context’s score is genuinely informed by the host-context features unique to it, rather than by the same flow-level features Temporal IForest already has access to. Both are tested in turn below.

3.3.1 Temporal IForest: Initial Assumption

If Temporal IForest is genuinely exploiting cross-timestep structure, its per-window score should remain informative about the *last* flow’s own label – the flow that would be paired with AE+Context’s per-flow score in the ensemble – rather than only about whether *some* flow within the 16-flow window is an attack. Its SHAP attribution should also not collapse onto that single last flow; if it does, the window is decorative and the “disjoint feature regime” argument for the ensemble does not hold.

3.3.2 Temporal IForest: Score Validity Under Last-Flow Labeling

The originally showed per-class AP figures use an “any-of-16” window label: a window is positive if any of its 16 flows is an attack, tagged with the majority attack type. That is more lenient than the per-flow labeling AE+Context is scored against. Every existing test window was relabeled by its *last* flow’s own true label instead, and per-class AP recomputed against the unchanged, already-trained model and its existing scores.

	AP (any-of-16)	AP (last-flow)	<i>n</i> (last-flow)
DDoS	0.9518	0.9515	95,123
DoS GoldenEye	0.6278	0.6265	7,567
DoS Slowhttptest	0.6851	0.6849	1,742
DoS slowloris	0.6091	0.6077	4,001
Heartbleed	0.5927	0.2892	11

We see AP is essentially unchanged for all four target classes under the stricter labeling, confirming the score is informative about the last flow’s own status, not an artifact of the

lenient window label. Heartbleed drops sharply, but on only 11 true flows – its entire test population – so it is excluded from the ensemble and treated as tentative, consistent with its hedged treatment in Chapter 2.

3.3.3 Temporal IForest: SHAP Attribution Across the Window

Temporal IForest is trained on flattened 16-flow windows, so `TreeExplainer`’s SHAP values on the flattened input decompose cleanly back into a (window, timestep, feature) cube. For each of the four classes, SHAP was computed on 200 windows; attribution magnitude was summed over features per timestep to check for concentration on the final timestep, and the top contributing features were plotted across all 16 timesteps to check whether the same feature carries consistent-sign attribution throughout the window rather than only near the end.

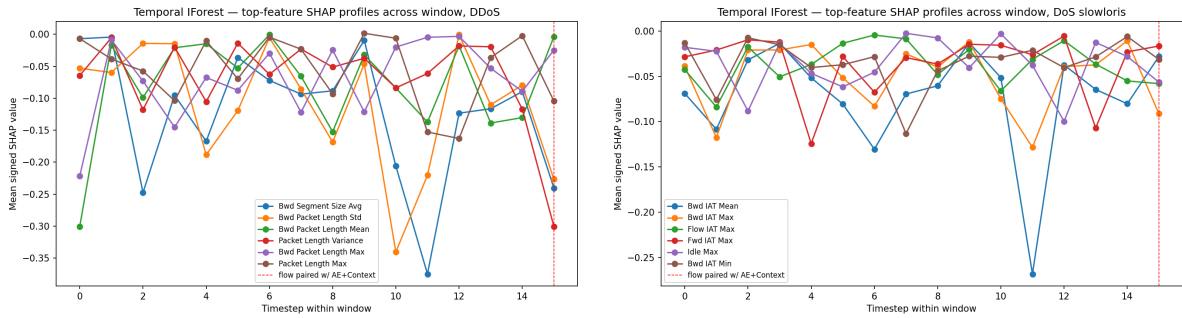


Figure 3.6: Temporal IForest – mean SHAP value per timestep for the six most-attributed features, DDoS windows (left) and DoS slowloris windows (right). In both cases attribution is negative (anomalous), recurs at consistent sign across most of the 16 timesteps, and does not concentrate at the final flow (dashed red line) – the flow paired with AE+Context’s score in the proposed ensemble.

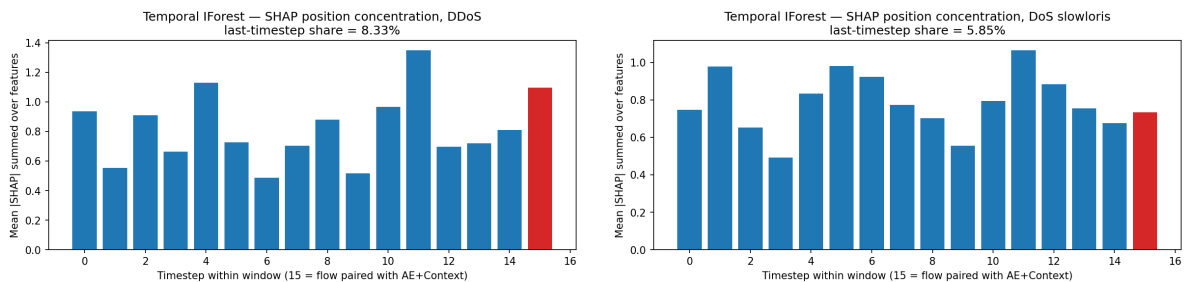


Figure 3.7: Temporal IForest – mean $|\text{SHAP}|$ summed over features, per timestep, DDoS windows (left) and DoS slowloris windows (right). The final timestep (red bar, 15) – the flow paired with AE+Context’s score in the proposed ensemble – carries no more attribution mass than any other position in the window, confirming the summary statistic reported below.

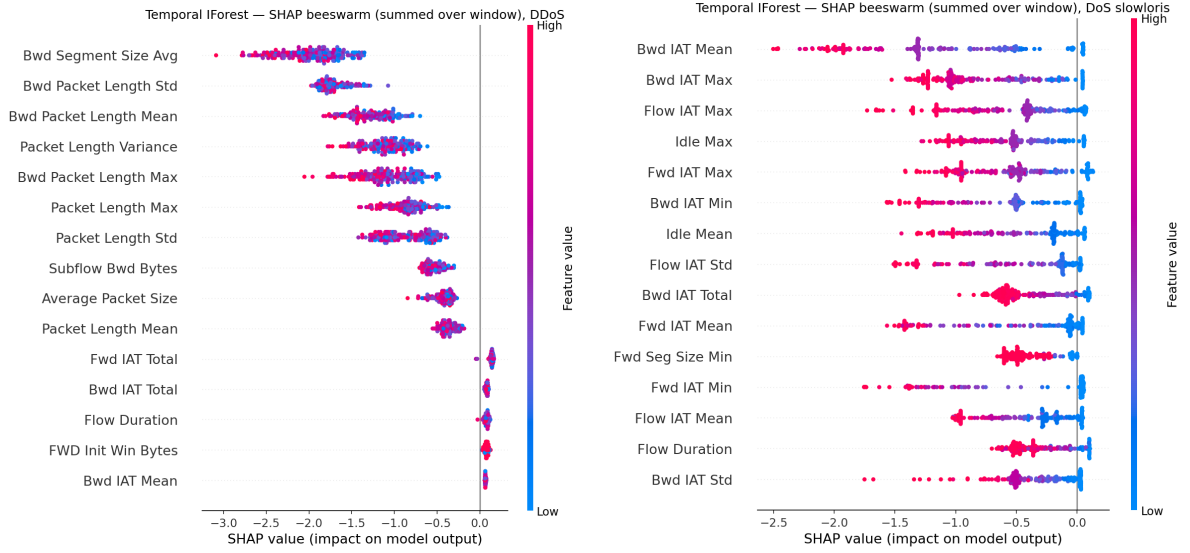


Figure 3.8: Temporal IForest – SHAP beeswarm per feature, DDoS windows (left) and DoS slowloris windows (right), collapsing the 16-timestep axis by summing each feature’s SHAP contribution across the window and coloring by that feature’s mean value across the window. DoS slowloris shows a clean monotonic relationship – high inter-arrival-time values drive strongly negative (anomalous) SHAP, low values push near zero – consistent with the slow, connection-holding behaviour the attack is named for. DDoS shows a noisier value-to-sign relationship for its backward-packet-length features, and a wider per-window spread than the mean-only view in Figure 3.6 suggests on its own.

The final timestep – the one paired with AE+Context in the ensemble – carries only 6–8% of total attribution mass across all four classes, close to the 6.25% uniform baseline for a 16-step window, rather than dominating it. The top features also match each attack’s known mechanism: DDoS and DoS GoldenEye attribution concentrates on backward-packet-length statistics (**Bwd Packet Length Std/Mean/Max**, **Packet Length Variance**), consistent with a volumetric flood; DoS Slowhttpstest and DoS slowloris attribution concentrates on inter-arrival-time statistics (**Fwd/Bwd IAT Std/Max**, **Idle Mean/Max**), consistent with the slow, connection-holding behaviour these attacks are named for. This confirms the initial assumption: Temporal IForest’s advantage on these classes reflects genuinely disjoint temporal evidence, not a relabeled copy of one flow’s static features – supporting the “disjoint feature regime” premise behind pairing it with AE+Context.

3.3.4 AE+Context: SHAP Dominated by Host-Context Features

The mirror premise is that AE+Context’s advantage on its own strong classes – DDoS and PortScan, per Chapter 2 – traces to the 9 `ctx_*` host-context features unique to its feature set, rather than to the same flow-level features Temporal IForest already sees. If instead

AE+Context’s attribution spreads across ordinary flow features, its complementarity with Temporal IForest would be coincidental rather than a genuinely disjoint evidence source.

SHAP was computed with `GradientExplainer` on AE+Context’s reconstruction error, on 200 samples each of PortScan and DDoS, and the 15 features with the highest mean $|\text{SHAP}|$ were ranked for each class.

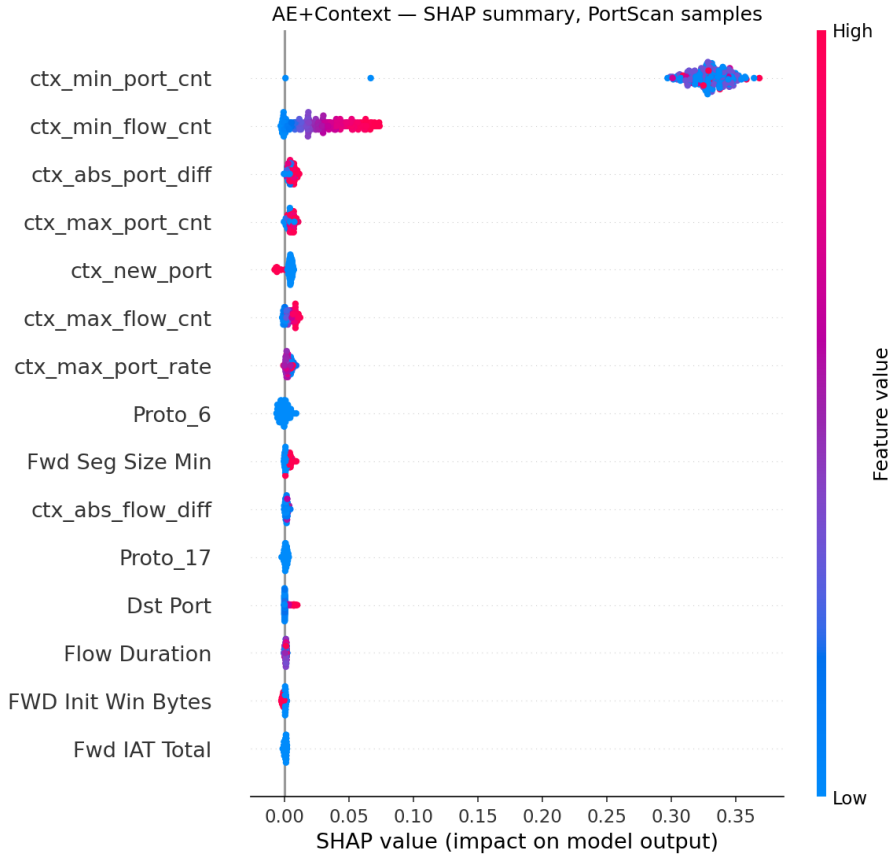


Figure 3.9: AE+Context – SHAP summary plot, PortScan samples. `ctx_min_port_cnt` dominates with a tight, uniformly positive-SHAP cluster well separated from every other feature; the remaining top rows are almost entirely `ctx_*` host-context features, with ordinary flow features only appearing near the bottom of the ranking.

	<code>ctx_*</code> features in top 15	Leading feature (mean $ \text{SHAP} $)
PortScan	8/15	<code>ctx_min_port_cnt</code> (0.324)
DDoS	7/15	<code>ctx_min_port_cnt</code> (0.234)

For both classes, `ctx_min_port_cnt` and `ctx_min_flow_cnt` dominate by roughly an order of magnitude over every ordinary flow feature: on PortScan, `ctx_min_port_cnt`’s attribution (0.324) is nearly $70\times$ the next-highest ordinary feature, `Fwd Seg Size Min` (0.0023); on DDoS it is over $6\times$ the leading ordinary feature, `Bwd Packet Length Std` (0.035). Six of the nine `ctx_*` features appear in both classes’ top-15 lists, with only `ctx_min_port_rate` absent from both. This confirms the mirror premise: AE+Context’s

advantage on these classes is driven almost entirely by host-context evidence unavailable to Temporal IForest, not by the ordinary flow features the two models share.

Taken together with §3.3.3’s result for Temporal IForest, the two halves of the disjoint-feature-regime premise both hold: Temporal IForest’s strong classes are explained by evidence spread across the 16-flow window (packet-length statistics for volumetric floods, inter-arrival-time statistics for slow-drip attacks), and AE+Context’s strong classes are explained by host-context features that exist only in its own feature set. The proposed ensemble’s complementary coverage is therefore backed by genuinely non-overlapping evidence between its two members, not by two rankings that merely happen to differ on which classes they cover.

3.3.5 Building the Combined Score

Confirming the disjoint-evidence premise does not by itself establish that combining the two scores works in practice. Realizing the ensemble sketched in Chapter 2 requires pairing, for every flow f in the test set, AE+Context’s own per-flow score $s_{AE}(f)$ with Temporal IForest’s score for the 16-flow window *ending* at f – the same causal, non-leaking choice validated in the previous subsection via last-flow labeling. Since the two preprocessing pipelines reorder flows differently before saving their respective test arrays (AE+Context sorts globally by timestamp; Temporal IForest sorts per source-IP host), the two score arrays are not aligned index-for-index and must be re-aligned by flow identity, recovering each flow’s original row position in both pipelines’ shared, pre-sort input.

Once aligned, the two raw scores cannot be combined directly: AE+Context’s score is a mean-squared reconstruction error with no fixed scale, and Temporal IForest’s score is a negated mean path length, and the two live on entirely different numeric ranges. Some normalization is required before either a weighted average or a max can be meaningful. Each model’s normalization statistics are computed on that same model’s own *benign training* scores only, never on test data, to avoid leaking test-set anomaly information into the fusion.

3.3.6 Two Fusion Rules, Two Normalizations

Two combination rules were tested. **Weighted soft voting** takes a convex combination of the two normalized scores, $s_{\text{soft}}(f) = w \cdot \hat{s}_{AE}(f) + (1 - w) \cdot \hat{s}_{IF}(f)$, swept over $w \in \{0.3, \dots, 0.7\}$. **Max-rule fusion** instead takes $s_{\text{max}}(f) = \max(\hat{s}_{AE}(f), \hat{s}_{IF}(f))$ – an OR-like combiner intended to match the disjoint-evidence story more directly: either model’s confidence alone should be enough to flag a flow.

The first normalization attempted, standard z -scoring against each model’s own benign training mean and standard deviation, turned out to be a dead end rather than a genuine test of either fusion rule. AE+Context’s reconstruction-error z -scores are extremely

heavy-tailed (maximum $z \approx 2366$ on the test set, with the 99.9th percentile of *benign* flows alone already at $z \approx 227$), while Temporal IForest’s z -scores stay tightly bounded (maximum $z \approx 21$). Under z -score normalization, a handful of extreme AE+Context outliers dominate any linear combination or max regardless of the weight assigned to Temporal IForest: max-rule and every soft-vote weight tested reproduced AE+Context’s own per-class ranking almost exactly (e.g. GoldenEye AP stayed at 0.215–0.216 across all five weights, against AE+Context alone’s 0.215), rather than showing any influence from Temporal IForest’s much stronger 0.627 standalone score on that class. This is a normalization artifact, not evidence that fusion does not help – it does not actually test the ensemble hypothesis.

Replacing z -score with rank normalization – mapping each test score to its percentile within that model’s own benign training-score distribution, via the empirical CDF – removes this problem by construction: every normalized score is bounded to $[0, 1]$ regardless of how extreme the raw value is, so neither model’s tail can dominate the other’s by sheer magnitude. All results below use rank normalization.

3.3.7 Per-Class Coverage vs. Aggregate AP

	DDoS	PortScan	GoldenEye	Slowhttpstest	slowloris
AE+Context alone	0.882	0.924	0.292	0.013	0.025
Temporal IForest alone	0.956	0.123	0.611	0.578	0.560
Max-rule (rank)	0.882	0.923	0.292	0.060	0.083
Soft-vote $w = 0.4$ (rank)	0.977	0.228	0.801	0.205	0.319
Soft-vote $w = 0.7$ (rank)	0.977	0.358	0.821	0.140	0.245

Aggregate AP over the full row-aligned test set actually *favours* the variants that change the least: max-rule (aggregate AP 0.943) and AE+Context alone (0.941) both edge out soft-vote $w = 0.4$ (0.841), simply because DDoS and PortScan together account for over 250,000 of the roughly 434,000 positive flows in this evaluation, while GoldenEye, Slowhttpstest, and slowloris combined account for barely 13,000. An aggregate metric dominated by two classes’ raw volume is exactly the wrong lens for judging whether an ensemble motivated by *per-class coverage* actually delivers it, so §2.3.2’s per-class breakdown, not the aggregate table, is the right basis for evaluating this ensemble – consistent with how every other per-attack-type claim in this thesis is judged.

Read per-class, max-rule falls short of its own premise: because it is an OR over two scores, it inherits the *union* of both models’ false-positive tendencies rather than only their true positives, so on GoldenEye/Slowhttpstest/slowloris it barely moves off AE+Context’s own weak scores ($0.292 \rightarrow 0.292$, $0.013 \rightarrow 0.060$, $0.025 \rightarrow 0.083$) despite Temporal IForest’s much stronger standalone numbers on exactly these classes. Soft voting shows the opposite failure mode on PortScan: because it averages rather than maxes,

and Temporal IForest is actively poor on PortScan (0.123, well below AE+Context’s 0.924) rather than merely uninformative, blending pulls PortScan’s ranking down substantially ($0.924 \rightarrow 0.228$ at $w = 0.4$) even while lifting GoldenEye from 0.292 to 0.801.

Neither rule is a free win, but soft voting is the more defensible choice for a coverage-motivated ensemble: its *weakest* covered class (0.205, Slowhttpptest, at $w = 0.4$) clears both standalone models’ own weakest classes – AE+Context’s 0.013 (Slowhttpptest) and Temporal IForest’s 0.123 (PortScan) – something neither standalone model nor max-rule fusion achieves. This is a genuine, if partial, realization of the ensemble idea from Chapter 2: rank-normalized soft voting measurably widens the floor of attack-class coverage, at the explicit cost of PortScan detection quality, rather than the free lunch the original aggregate-level framing implied. A deployment that must retain AE+Context’s near-complete PortScan coverage would instead need a class-aware combination rule (e.g. routing PortScan-like flows to AE+Context alone) rather than a single global weight – left as future work.